Raport projektowy
Uczenie się ze wzmocnieniem

Alina Skyba

2026/2027

Spis treści

1 Informacje podstawowe **2**

2 Temat i cel projektu **2**
2.1 Opis problemu 2
2.2 Cel agenta 2
2.3 Uzasadnienie zastosowania uczenia się ze wzmocnieniem 2

3 Formalizacja problemu RL/MDP **3**
3.1 Agent i środowisko 3
3.2 Stan lub obserwacja 3
3.3 Akcje 3
3.4 Funkcja nagrody 4
3.5 Warunki zakończenia epizodu 4
3.6 Założenia i ograniczenia środowiska 4

1 Informacje podstawowe

Element	Informacja
Imię i nazwisko	Alina Skyba
Numer albumu	181628
Kierunek/specjalność	Informatyka/AI
Tytuł projektu	Uczenie agenta walki w pojedynku
Link do repozytorium	https://github.com/akmqu/rl-duel-agent
Data oddania	07.11.2026

2 Temat i cel projektu

2.1 Opis problemu

Projekt dotyczy prostej gry 2D (widok z boku), w której dwóch samurajów walczy ze sobą na jednej linii. Wystarczy jedno trafienie mieczem, żeby pokonać przeciwnika, więc wynik zależy od tego, czy zawodnik podejmie dobrą decyzję we właściwym momencie. Grę zaimplementuję samodzielnie w Pythonie.

Agent steruje jednym samurajem, a jego przeciwnikiem jest bot działający według prostego skryptu. W każdej chwili agent może się ruszyć do przodu lub do tyłu, uderzyć, zablokować cios, zrobić unik albo nic nie robić. Cios ma trzy części: przygotowanie, samo uderzenie i odnowę. Podczas odnowy zawodnik nie może się bronić, dlatego chybiony cios można wykorzystać do kontrataku.

2.2 Cel agenta

Celem agenta jest wygrać pojedynek, czyli trafić przeciwnika wcześniej, niż on trafi agenta. Żeby to osiągnąć, agent powinien nauczyć się:

- utrzymywać odpowiednią odległość od przeciwnika, tak aby być poza jego zasięgiem, ale blisko,
- wykorzystywać chybienia przeciwnika i atakować, gdy ten nie może się bronić,
- reagować blokiem lub unikiem na cios przeciwnika.

Za skuteczne działanie agenta uznam sytuację, w której wygrywa on częściej niż agent bazowy.

2.3 Uzasadnienie zastosowania uczenia się ze wzmocnieniem

- W każdej chwili agent podejmuje decyzję, a dobra strategia to ciąg takich decyzji (np. cofnąć się, poczekać na cios, zrobić unik i kontratakować), a nie jedna decyzja.
- Decyzja ma wpływ na to, co może się wydarzyć potem. Po wyprowadzeniu ciosu agent przez kilka taktów nie może się bronić, więc zły moment ataku kończy się porażką.
- Nagrodę dostaje się dopiero na końcu pojedynku, ale dobre decyzje (np. trzymanie dystansu) zapadają dużo wcześniej. Agent musi więc uczyć się, które z nich naprawdę się opłacają.

- Część przeciwników działa w sposób losowy, a agent nie wie, co przeciwnik zrobi w następnym kroku.
- To nie jest zwykła klasyfikacja ani jednorazowa optymalizacja. Agent wykonuje wiele ruchów jeden po drugim, a każdy z nich wpływa na dalszy przebieg walki.

3 Formalizacja problemu RL/MDP

3.1 Agent i środowisko

Agentem jest lewy samuraj. Środowiskiem jest arena (linia o określonej długości), prawy samuraj sterowany skryptem oraz zasady walki. Czas płynie w krokach.

W jednym kroku agent wybiera akcję, przeciwnik wybiera swoją, a następnie środowisko sprawdza, czy ktoś kogoś trafił, i zwraca agentowi nowy stan, nagrodę oraz informację, czy pojedynek się skończył.

Przeciwnicy sterowani skryptem będą trzech rodzajów: agresywny (podchodzi i atakuje), obronny (blokuje ciosy i kontratakuje) oraz losowy (w bliskiej odległości atakuje lub się cofa, z określonym prawdopodobieństwem). Agenta uczącego się chcę trenować na dwóch z nich, a sprawdzić na trzecim, którego wcześniej nie widział.

Agenci bazowi, z którymi porównam agenta uczącego się: agent losowy, agent, który zawsze atakuje, oraz prosta heurystyka (utrzymuje dystans, blokuje cios, a po chybieniu przeciwnika atakuje).

3.2 Stan lub obserwacja

Agent widzi trzy rzeczy: odległość od przeciwnika, to, co sam aktualnie robi, oraz to, co robi przeciwnik (np. przygotowuje cios, jest w odnowie). Nie widzi natomiast, co przeciwnik zamierza zrobić, więc nie zna pełnego stanu środowiska.

Składowa obserwacji	Znaczenie
Odległość	ile pól dzieli samurajów
Faza własna	co agent robi w tej chwili (np. stoi, przygotowuje cios, blokuje)
Faza przeciwnika	co w tej chwili robi przeciwnik (np. stoi, przygotowuje cios, jest w odnowie)

Liczba wszystkich możliwych stanów wynosi zaledwie kilka tysięcy, dlatego na początek planuję użyć klasycznego algorytmu Q-learning.

3.3 Akcje

Agent ma do wyboru sześć akcji.

Akcja	Opis
Nic	agent czeka
Krok do przodu	zbliża się do przeciwnika o jedno pole
Krok do tyłu	oddala się od przeciwnika o jedno pole
Cios	atak mieczem (przygotowanie, uderzenie, odnowa)
Blok	obrona przed ciosem; zablokowany cios zatrzymuje atakującego
Unik	krótka nietykalność i cofnięcie się o jedno pole

Nową akcję można zacząć dopiero wtedy, gdy poprzednia się skończy. Jeśli agent wybierze akcję w złym momencie, nic się nie dzieje. Nie można też wyjść poza arenę ani przejść przez przeciwnika.

3.4 Funkcja nagrody

- $+1$ za trafienie przeciwnika (wygrana),
- -1 za bycie trafionym (porażka),
- $-0{,}001$ za każdy krok.

Wygrana jest jedynym celem, dlatego dostaje największą nagrodę, a porażka równie dużą karę. Mała kara za krok sprawia, że agent nie wydłuża pojedynku bez końca. Remis (koniec czasu) jest więc gorszy od wygranej, ale lepszy od porażki. Agent może się nauczyć unikać walki, żeby skończyć remisem zamiast ryzykować porażkę. Sprawdzę to w eksperymentach. Planuję też porównać kilka wersji nagród.

3.5 Warunki zakończenia epizodu

Epizod kończy się, gdy agent trafi przeciwnika (wygrana), gdy przeciwnik trafi agenta (porażka) albo gdy minie limit 200 kroków (remis).

3.6 Założenia i ograniczenia środowiska

- Arena to prosta linia, a gra toczy się krok po kroku.
- Uczy się tylko jeden agent, a przeciwnicy działają według skryptów.
- Losowość jest w zachowaniu części przeciwników oraz w tym, że na początku stoją w różnej odległości od siebie.
- Czas trwania poszczególnych części ciosu i zasięg miecza są na razie wstępne. Mogę je zmienić po pierwszych testach.